



Regex Cheat Sheet

Full Stack Tutorial visit thedevspace.io.

`/ ( reg ) / ex`

1 Regex Basics

Regex (Regular Expressions) help you search, match, and manipulate text patterns in JavaScript. You can create Regex using literals (`/<pattern>/<flag>`) or the `RegExp` constructor.

```
1 const regex = /hello/i; // literal
2 const regex2 = new RegExp("hi", "g"); //
  constructor
```

🚩 Common Regex Flags

Flags modify the behavior of a regex, such as global search, case-insensitivity, and more.

```
1 /g      // Global match
2 /i      // Case-insensitive
3 /m      // Multiline
4 /s      // Dot matches newline
5 /u      // Unicode
6 /y      // Sticky match
```



Character Classes

These match specific types of characters like digits, whitespace, or word characters.

```
1 .          // Any character except newline
2 \d         // Digit (0-9)
3 \D         // Not a digit
4 \w         // Word character (a-z, A-Z, 0-9,
               _)
```

```
5 \W         // Not a word character
6 \s         // Whitespace
7 \S         // Not whitespace
```

12
34

Quantifiers

Quantifiers control how many times a pattern can occur.

```
1 *      // 0 or more
2 +      // 1 or more
3 ?      // 0 or 1
4 {n}    // Exactly n
5 {n,}   // At least n
6 {n,m}  // Between n and m
```



Anchors

Anchors match positions like the start or end of a string.

```
1 ^      // Start of string
2 $      // End of string
3 \b     // Word boundary
4 \B     // Non-word boundary
```

Groups and Ranges

Group patterns and define custom character ranges.

```
1 (a|b)    // a or b
2 [abc]    // a, b, or c
3 [^abc]   // Not a, b, or c
4 [a-z]    // Range from a to z
```

Lookahead & Lookbehind

Used to match patterns based on what comes before or after them.

```
1 (?=y)    // Positive lookahead
2 (?!y)    // Negative lookahead
3 (?≤x)    // Positive lookbehind
4 (?<!x)   // Negative lookbehind
```

2 Built-in Methods

Testing with `.test()`

Use `.test()` to check if a pattern exists in a string. Returns either `true` or `false`.

```
1 const regex = /dog/;  
2 regex.test("my dog is cute"); // true
```

Extracting with `.match()`

`.match()` retrieves the matches from a string. Returns all matches when used with the `g` flag.

```
1 "hello 123".match(/\d+/); // ["123"]  
2 "hi 123 hi 456".match(/\d+/g); //  
  ["123", "456"]
```

Using `.exec()` for advanced matching

`.exec()` gives detailed match results, useful in loops for repeated capturing.

```
1 const regex = /\d+/g;
2 let match;
3 while ((match = regex.exec("1 fish 2
  fish 3 fish")) !== null) {
4   console.log(match[0]); // "1", then
    "2", then "3"
5 }
```

Replacing with `.replace()`

`.replace()` method replaces matched patterns with a new value. Supports named replacement group (`$<Name>`) for capturing and replacing groups.

```
1 "cat".replace(/c/, "b"); // "bat"
2 "2025-04-18".replace(/(\d{4})-(\d{2})-
  (\d{2})/, "$3/$2/$1"); // "18/04/2025"
```

🪄 Splitting with `.split()`

Split a string by a regex pattern instead of a fixed string.

```
1 "apple,banana orange".split(/[ ,]+/);  
  // ["apple", "banana", "orange"]
```

🔍 Searching with `.search()`

`.search()` returns the index of the first match, or `-1` if none is matched.

```
1 "find the match".search(/match/); // 9
```


3 Practice Examples

Matching a Set of Characters

Use square brackets `[]` to match any one character in a set.

```
1 /[aeiou]/.test("apple"); // true  
  (matches "a")  
2 /[0-9]/.test("year 2025"); // true  
  (matches "2")
```

Excluding a Set of Characters

Use `[^...]` to match anything except the listed characters.

```
1 /^[^0-9]/.test("abc"); // true (matches  
  "a")  
2 /^[^aeiou]/.test("apple"); // true  
  (matches "p")
```



Start and End Anchors

Use `^` and `$` to match the beginning or end of a string.

```
1 /^Hello/.test("Hello world"); // true
2 /world$/.test("Hello world"); // true
3 /^Hi$/.test("Hi"); // true (exact match)
```



Word Boundary

`\b` asserts a word boundary (edge between word and non-word), and `\B` asserts non-boundary.

```
1 /\bcat/.test("a cat"); // true (starts
  at word boundary)
2 /cat\b/.test("black cat"); // true (ends
  at word boundary)
3 /\Bcat/.test("concatenate"); // true
  (not at word boundary)
```



Quantifiers

Control how many times a pattern should appear.

```
1 /a*/.test(""); // true (0 or more 'a')
2 /a+/.test("aa"); // true (1 or more)
3 /a?/.test("b"); // true (0 or 1)
4 /a{3}/.test("aaa"); // true (exactly 3)
5 /a{2,4}/.test("aaa"); // true (between 2
and 4)
6 /a{2,}/.test("aaaa"); // true (2 or
more)
```



Capturing Groups

Use parentheses to capture matched groups. Use `.match()` or `.exec()` to retrieve them.

```
1 const result = /(\d+)-(\d+)-
  (\d+)/.exec("2025-04-18");
2 console.log(result[1]); // "2025"
3 console.log(result[2]); // "04"
4 console.log(result[3]); // "18"
```



Alternation

Use the pipe `|` to match either/or conditions.

```
1 /yes|no/.test("no"); // true
2 /apple|banana|cherry/.test("banana"); //
  true
```



Lookahead and Lookbehind

Check if something is (or isn't) ahead or behind, without including it in the match.

```
1 // Lookahead
2 /\d(?:px)/.exec("10px"); // matches "0"
  (followed by "px")
3 /\w+(?:\.)/.exec("test.js"); // matches
  "test"
4
5 // Negative lookahead
6 /\d(?:!px)/.exec("10em"); // matches "0"
  (not followed by "px")
7
8 // Lookbehind
9 /(?:<=\$)\d+/.exec("$50"); // matches
  "50"
10 /(?:<!\$)\d+/.exec("50"); // matches
   "50" (not preceded by $)
```

Real-World Applications

Find All Vowels in a Word

```
1 "banana".match(/[aeiou]/g); // ["a",  
  "a", "a"]
```

Match Words Regardless of Case

```
1 "Color".match(/[cC]olor/); // ["Color"]
```

Remove All Digits from a String

```
1 "abc123".match(/^[^0-9]/g); // ["a", "b",  
  "c"]
```

Find Non-Letter Symbols in Input

```
1 "safe!".match(/[^\a-z]/gi); // ["!"]
```



Check if URL Starts with http or https

```
1 /^https?:\/\/\./.test("https://example.com"); // true
```



Verify File Extension (e.g. .js)

```
1 /\.js$/.test("app.min.js"); // true
```



Match Whole Words Only

```
1 /\bcat\b/.test("The cat sat"); // true
2 /\bcat\b/.test("concatenate"); // false
```



Extract Zip Code from Text

```
1 "zip code: 12345".match(/\d{5}/); // ["12345"]
```



Normalize Repeated Characters

```
1 "haaa".match(/ha{2,}/); // ["haa"]
```



Reformat a Date String

```
1 "2025-04-18".replace(/(\d{4})-(\d{2})-(\d{2})/, "$3/$2/$1");  
2 // "18/04/2025"
```



Match Repeated Word Segments

```
1 "foofoo".match(/(?:foo)+/); //  
  ["foofoo"]
```



Support British and American Spelling

```
1 /color|colour/.test("My favorite  
  color"); // true
```




Find Either "dog" or "cat"

```
1 "dog or cat".match(/dog|cat/); //  
  ["dog"]
```



Extract Numeric Value Before "px"

```
1 "100px".match(/\d+(?=px)/); // ["100"]
```



Extract Amount After Dollar Sign

```
1 "$50".match(/(?<=\$)\d+/); // ["50"]
```

Exclude Amounts Already with \$ Sign

```
1 "50".match(/(?<!\$)\d+/); // ["50"]
```

Match Numbers Not Followed by "px"

```
1 "100em".match(/\d+(?!px)/); // ["100"]
```